Índice

1. Introducción **2**
1.1. Planteamiento del Problema 2
1.2. Motivación 2

2. Objetivos **2**
2.1. Objetivo General 2
2.2. Objetivos Específicos 2

3. Marco Teórico **3**
3.1. Fundamentos del Polimorfismo y Tipos de Invocación 3
3.2. Clases Abstractas vs. Interfaces en Java 3
3.3. Polimorfismo con Estructuras Dinámicas (ArrayList de Interfaz) 3

4. Desarrollo **3**
4.1. Actividad 1: Consulta Técnica con IA sobre la Elaboración de una GUI para Login 3
4.2. Actividad 2: Implementación de la Interfaz HuellaCarbono y Clases Dispares . . 4
4.3. Diagrama de Clases UML 4
4.4. Casos de Prueba y Metodología de Validación 5

5. Resultados **6**
5.1. Consultas a IAs por la Brigada 6
5.2. Evidencias de Ejecución en Terminal 6

6. Conclusiones **7**

7. Referencias **7**

1. Introducción

1.1. Planteamiento del Problema

En el modelado de software orientado a objetos, con frecuencia se requiere procesar un conjunto homogéneo de comportamientos sobre entidades que no comparten una jerarquía de herencia directa ni atributos en común. Intentar unificar mediante herencia tradicional clases dispares, como construcciones edilicias, vehículos de combustión y medios de transporte no motorizados, genera jerarquías forzadas y poco coherentes que rompen la semántica del diseño. En esta práctica resolvemos dicha problemática mediante la implementación de una interfaz Java denominada `HuellaCarbono`, la cual define un contrato de comportamiento abstracto a través del método `getHuellaCarbono()`. Este método es implementado de forma personalizada por tres clases heterogéneas: `Edificio`, `Auto` y `Bicicleta`, que calculan sus respectivas emisiones.

Posteriormente, mediante una colección dinámica de tipo `ArrayList<HuellaCarbono>`, se itera sobre los objetos invocando polimórficamente sus métodos para determinar el impacto ambiental individual.

1.2. Motivación

El polimorfismo constituye uno de los pilares más potentes de la programación orientada a objetos (POO), ya que permite enviar mensajes sintácticamente idénticos a objetos de distintos tipos y obtener ejecuciones adaptadas a cada contexto. Comprender la diferencia entre el acoplamiento rígido de la herencia de clases y la flexibilidad de las interfaces nos capacita para diseñar sistemas extensibles y desacoplados. Las interfaces representan contratos de comportamiento y permiten tratar a múltiples clases bajo un mismo tipo de referencia abstracto, posibilitando la iteración polimórfica en colecciones dinámicas y facilitando la mantenibilidad a largo plazo.

2. Objetivos

2.1. Objetivo General

Implementar el concepto de polimorfismo en un lenguaje de programación orientado a objetos.

2.2. Objetivos Específicos

- Diseñar la interfaz `HuellaCarbono` con la firma del método `getHuellaCarbono()` para establecer un contrato de comportamiento uniforme.
- Implementar las clases independientes `Edificio`, `Auto` y `Bicicleta`, agregar atributos específicos de su dominio y realizar la implementación concreta de la interfaz.
- Investigar, mediante herramientas de inteligencia artificial generativa, los factores de emisión requeridos para calcular matemáticamente la huella de carbono de cada entidad.
- Construir una clase principal `Main` que cree instancias de las clases desarrolladas, las almacene en un `ArrayList<HuellaCarbono>` y recorra la lista invocando el método de forma polimórfica.
- Consultar herramientas de inteligencia artificial generativa sobre la elaboración de interfaces gráficas de usuario (GUI) para módulos de autenticación (login), analizando los enfoques de diseño de formularios.

3. Marco Teórico

3.1. Fundamentos del Polimorfismo y Tipos de Invocación

Etimológicamente, el término polimorfismo proviene del griego y significa «muchas formas»[1]. En programación orientada a objetos, se refiere a la propiedad por la cual es posible enviar mensajes sintácticamente iguales a objetos de distintos tipos[1]. El polimorfismo se clasifica en dos categorías principales[1]:

1. **Polimorfismo estático (ad hoc):** se resuelve en tiempo de compilación. Los tipos de datos sobre los que se opera deben especificarse explícitamente antes de ejecutarse; un ejemplo es la sobrecarga de métodos[1].

2. **Polimorfismo dinámico (de subtipo o tiempo de ejecución):** se resuelve durante la ejecución. Una variable de referencia de un tipo abstracto o superclase puede apuntar a instancias de sus subclases o implementaciones, y el método correspondiente se selecciona durante la ejecución del programa[1].

3.2. Clases Abstractas vs. Interfaces en Java

Aunque ambas estructuras impiden la instanciación directa de objetos, presentan diferencias conceptuales importantes en el modelado del dominio:

- **Clases abstractas:** representan una relación taxonómica *IS-A* («es un»). Pueden contener estados (atributos con distintos modificadores), métodos concretos con implementación y métodos abstractos. Una clase únicamente puede heredar de una sola clase debido a la restricción de herencia simple de Java[2].

- **Interfaces:** representan un contrato de comportamiento o capacidad (*CAN-DO*). Sus métodos declarados son públicos y abstractos de manera implícita, salvo los casos que Java permite implementar mediante métodos `default` o estáticos. Las interfaces permiten definir capacidades compartidas por clases que no necesitan tener un ancestro común[2].

3.3. Polimorfismo con Estructuras Dinámicas (ArrayList de Interfaz)

Una variable de referencia declarada con un tipo de interfaz puede referenciar cualquier objeto cuya clase implemente dicha interfaz[3]. Al combinar este principio con colecciones genéricas como `ArrayList<HuellaCarbono>`, es posible estructurar colecciones dinámicas heterogéneas. Al iterar sobre la colección, la máquina virtual de Java (JVM) ejecuta el enlace dinámico (*dynamic binding*), invocando la versión concreta del método redefinido en cada objeto.

4. Desarrollo

4.1. Actividad 1: Consulta Técnica con IA sobre la Elaboración de una GUI para Login

Para dar cumplimiento al primer ejercicio de la práctica, cada integrante de la brigada consultó una herramienta de inteligencia artificial generativa diferente (ChatGPT, Claude, Gemini, Copilot y DeepSeek), solicitando orientación sobre la elaboración de un formulario GUI para un sistema de inicio de sesión con usuario y contraseña.

El análisis de las propuestas resalta que, en el ecosistema Java, la creación de interfaces gráficas de usuario suele abordarse mediante bibliotecas como Swing (`JFrame`, `JTextField`, `JPasswordField`, `JButton`) o tecnologías más modernas como JavaFX (`Stage`, `Scene`, `VBox`,

`TextField`). Los aspectos clave para la construcción técnica de esta interfaz incluyen el uso correcto de gestores de diseño (*Layout Managers*, como `GridBagLayout` o `MigLayout`), la captura de eventos mediante escuchadores (*Event Listeners* o expresiones lambda asociadas al botón de envío) y la gestión segura de credenciales. En este último aspecto, las contraseñas deben procesarse mediante arreglos de caracteres `char[]` obtenidos de `JPasswordField`, en lugar de cadenas `String` inmutables, para reducir su permanencia en la memoria RAM.

4.2. Actividad 2: Implementación de la Interfaz HuellaCarbono y Clases Dispares

Se diseñó e implementó un sistema orientado a objetos para cuantificar las emisiones de CO_2 mediante el uso de interfaces y polimorfismo dinámico:

1. **Interfaz `HuellaCarbono`:** define la firma del método abstracto `getHuellaCarbono()`, que retorna un valor decimal de tipo `double`, correspondiente a los kilogramos de CO_2 producidos.

2. **Clase `Edificio`:** modela una estructura edilicia. Incorpora atributos privados para el nombre, el consumo de energía eléctrica en kilovatios-hora (`consumoKwh`) y el consumo de gas en metros cúbicos (`consumoGasM3`). La implementación de `getHuellaCarbono()` multiplica el consumo eléctrico por un factor de **0.42 kg CO_2/kWh** y el consumo de gas por **2.0 kg CO_2/m^3**.

3. **Clase `Auto`:** modela un vehículo automotor. Contiene atributos privados para la matrícula, los kilómetros recorridos y el rendimiento en kilómetros por litro. Su método `getHuellaCarbono()` calcula los litros de combustible consumidos (dividiendo los kilómetros recorridos entre el rendimiento) y los multiplica por un factor de emisión de **2.31 kg CO_2/litro** de gasolina.

4. **Clase `Bicicleta`:** modela un medio de transporte no motorizado. Almacena el número de serie y los kilómetros recorridos. Su método `getHuellaCarbono()` retorna un valor constante de **0.0 kg CO_2**, reflejando la ausencia de emisiones directas durante su uso.

5. **Clase principal `Main`:** instancia objetos de las tres clases anteriores, los añade a una colección dinámica de tipo `ArrayList<HuellaCarbono>` y ejecuta un bucle iterativo que invoca polimórficamente el método `getHuellaCarbono()`, mostrando la descripción de cada entidad mediante `toString()` y su correspondiente impacto ambiental.

4.3. Diagrama de Clases UML

La Figura 1 resume la relación entre la clase principal, la interfaz compartida y las tres clases que implementan el cálculo de la huella de carbono.

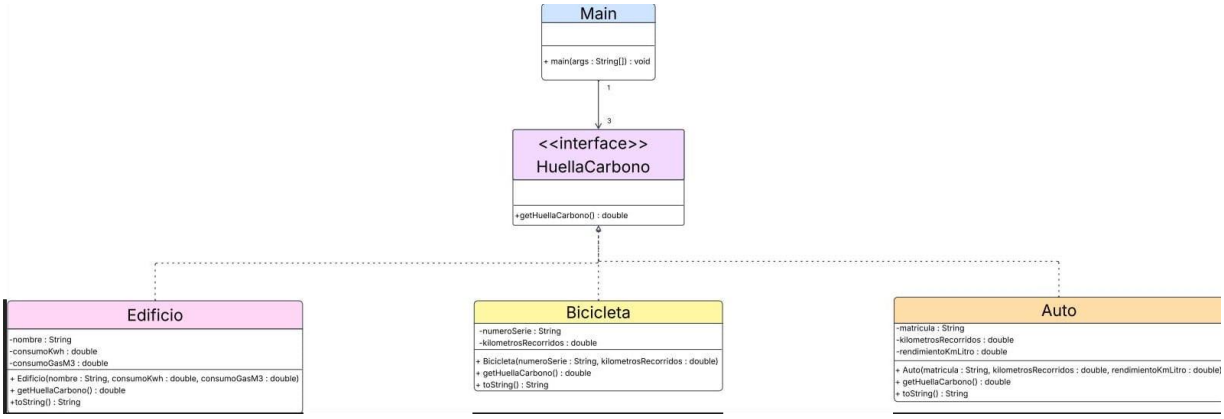


Figura 1. Diagrama de clases UML del sistema para el cálculo de Huella de Carbono.

4.4. Casos de Prueba y Metodología de Validación

Metodología de pruebas

Se validaron las implementaciones de las tres clases mediante los datos de entrada empleados en la ejecución del programa. Las comprobaciones permiten contrastar las fórmulas definidas en el código con el resultado de cada invocación polimórfica:

1. **Caso 1 (Edificio):** para el edificio *Torre Central FI*, se utilizaron 4500.0 kWh de electricidad y 180.0 m^3 de gas. La operación $(4500.0 \times 0.42) + (180.0 \times 2.0)$ produce una huella de **2250.0 kg CO_2**.

2. **Caso 2 (Auto):** para el automóvil con matrícula `PXZ-4821`, se consideraron 1200.0 km recorridos y un rendimiento de 14.5 km/l. La operación $(1200.0/14.5) \times 2.31$ produce aproximadamente **191.17 kg CO_2**. La terminal muestra el valor de tipo `double` con mayor precisión.

3. **Caso 3 (Bicicleta):** para la bicicleta con número de serie `BIC-2026-X8`, se registraron 350.0 km recorridos. El método retorna de forma constante **0.0 kg CO_2**, ya que el modelo no contabiliza emisiones directas por su uso.

Tabla 1. Casos de prueba y resultados de la ejecución.

Caso / entidad	**Datos de entrada**	**Fórmula aplicada**	**Huella calculada**
Edificio	Nombre: `Torre Central FI`; consumo de luz: 4500.0 kWh; consumo de gas: 180.0 m^3.	$(4500.0 \times 0.42) + (180.0 \times 2.0)$	2250.0 kg CO_2
Auto	Matrícula: `PXZ-4821`; kilómetros recorridos: 1200.0 km; rendimiento: 14.5 km/l.	$(1200.0/14.5) \times 2.31$	191.17 kg CO_2
Bicicleta	Serie: `BIC-2026-X8`; kilómetros recorridos: 350.0 km.	Retorno constante de 0.0	0.0 kg CO_2

Nota: el programa imprime 191.17241379310346 kg CO2 para el automóvil; la tabla lo redondea a 191.17 kg CO2. Edificio y Bicicleta coinciden con la terminal.

5. Resultados

5.1. Consultas a IAs por la Brigada

A continuación se recopilan las consultas técnicas realizadas por los cinco miembros de la brigada sobre el diseño de interfaces gráficas para módulos de autenticación (Ejercicio 1):

Integrante 1: Bazán Pantaleón Rosa María | Herramienta: ChatGPT (OpenAI)

Prompt: «¿Cómo puedo elaborar una interfaz gráfica de usuario (GUI) en Java Swing para un formulario de login con usuario y contraseña?»

Respuesta: «Para construir un login en Java Swing se utiliza un `JFrame` como ventana principal, organizado mediante un `JPanel` con `GridBagLayout` para alinear los componentes. Se agregan dos `JLabel` para "Usuario" y "Contraseña", un `JTextField` para el nombre, un `JPasswordField` para ocultar la clave y un `JButton` para autenticar. El evento del botón se gestiona con `addActionListener`.»

Integrante 2: García Rivera Gabriel Eduardo | Herramienta: Copilot (Microsoft)

Prompt: «¿Qué componentes de Java Swing son necesarios para hacer una pantalla de autenticación de usuario?»

Respuesta: «Se necesita un `JFrame` que actúe como contenedor base y un `JPanel` para la distribución. Se agregan dos objetos `JLabel` como identificadores de texto, un `JTextField` para el usuario, un `JPasswordField` para la contraseña y un `JButton` que implemente un `ActionListener` para desencadenar la verificación.»

Integrante 3: Padrón López María Fernanda Corina | Herramienta: Gemini (Google)

Prompt: «Muestra la arquitectura recomendada para crear un login con Swing en Java usando buenas prácticas.»

Respuesta: «Se debe instanciar un `JFrame` configurando `setDefaultCloseOperation( JFrame.EXIT_ON_CLOSE)`. Dentro del panel, se posicionan componentes como `JTextField` y `JPasswordField`. Para capturar los datos, se lee el texto mediante `getText()` para el usuario y `getPassword()` en el campo seguro, procesando la validación en un método independiente.»

Integrante 4: Toledo Valencia Aimee Guadalupe | Herramienta: Claude (Anthropic)

Prompt: «Explica las mejores prácticas para estructurar una interfaz gráfica de login en Java Swing de forma limpia y segura.»

Respuesta: «Estructuralmente, se recomienda separar la lógica del controlador de la vista. Para la vista, se utiliza `JPasswordField` en lugar de `JTextField` para evitar almacenar contraseñas como cadenas inmutables en memoria. Se recomienda emplear un gestor de diseño como `GroupLayout` o `MigLayout` para mantener la proporcionalidad durante el redimensionamiento.»

Integrante 5: Vitla Herrera Cesar Antonio | Herramienta: DeepSeek

Prompt: «Proporciona el diseño conceptual de una interfaz de login en Java evitando errores de seguridad y diseño.»

Respuesta: «Diseña el formulario agrupando controles en un `JPanel`. Es importante usar `JPasswordField.getPassword()`, ya que devuelve un arreglo `char[]` que se puede limpiar de la memoria inmediatamente tras validar, a diferencia de un `String`, que permanece en el recolector de basura.»

5.2. Evidencias de Ejecución en Terminal

La Figura 2 muestra la salida del programa para las tres clases.

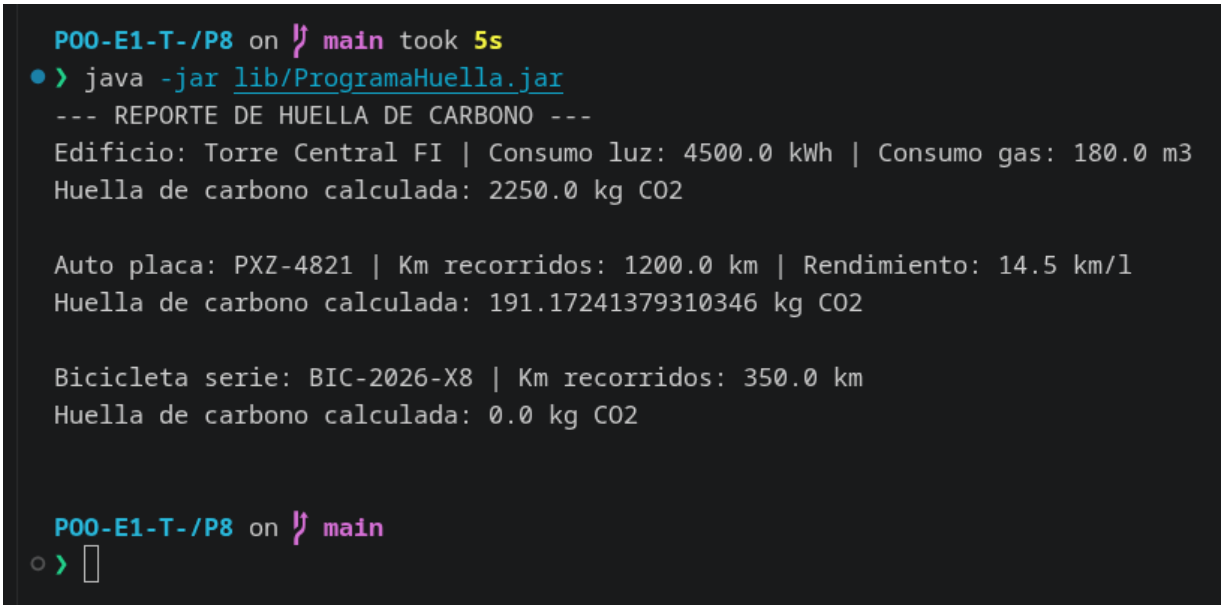


Figura 2. Ejecución en terminal que muestra el cálculo de la huella de carbono para las tres clases.

6. Conclusiones

La realización de esta práctica permitió comprobar la importancia del polimorfismo dinámico y el uso de interfaces para resolver problemas de diseño en los que interactúan clases heterogéneas. Al definir la interfaz `HuellaCarbono`, se estableció un contrato de comportamiento uniforme que permitió vincular clases sin relación taxonómica previa, como `Edificio`, `Auto` y `Bicicleta`, superando las limitaciones de la herencia tradicional. Asimismo, la integración de una colección dinámica de tipo `ArrayList<HuellaCarbono>` demostró la utilidad del enlace dinámico en Java, al permitir almacenar referencias abstractas e invocar polimórficamente el método `getHuellaCarbono()` para ejecutar el cálculo correspondiente de cada objeto durante la ejecución. Finalmente, el análisis sobre el desarrollo de interfaces gráficas para autenticación enriqueció el criterio arquitectónico de la brigada respecto a la separación de responsabilidades y el manejo seguro de los datos de entrada en el desarrollo de aplicaciones robustas.

7. Referencias

[1] J. A. Solano, «Polimorfismo», Unidades de Apoyo para el Aprendizaje, CUAED/Facultad de Ingeniería-UNAM, 2020. [En línea]. Disponible en: https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/3062/mod_resource/content/1/UAPA-Polimorfismo/index.html

[2] Equipo Geek - NTT DATA, «Clases abstractas e interfaces: ¿Qué son?», ifGeekThen - NTT DATA, 19 de septiembre de 2019. [En línea]. Disponible en: https://ifgeekthen.nttdata.com/s/post/clases-abstractas-e-interfaces-MCRLZG2CKOIFF2JKPMO7XLKRWJAI?language=es

[3] Á. López, «Polimorfismo en Java: tipos, ejemplos y ejercicios», Ciberaula, marzo de 2026. [En línea]. Disponible en: https://www.ciberaula.com/cursos/java/polimorfismo.php#interfaces

	Carátula para entrega de prácticas
Facultad de Ingeniería	Laboratorio de docencia

Laboratorio de Computación Salas A y B

Profesor(a): M.C. René Adrián Dávila Pérez

Asignatura: Programación orientada a objetos

Grupo: 07

No de práctica(s): 08

Integrante(s): 424045919 | 323087067 | 426060033 | 426059961 | 323297930

No de lista o brigada: 1

Semestre: 2027 - 1

Fecha de entrega: 23 de octubre de 2026

Observaciones:

Calificación: